

Image to Garment Pattern Generation for a Local Backend System

Executive summary

If your target is **accurate, metric, sewable patterns** rather than pretty meshes, the field is converging on one clear idea: **structured pattern representations beat raw geometry generation**. The strongest systems either generate a **parametric garment language** such as GarmentCode or a **structured JSON/DSL layer** that can be deterministically compiled into panels and stitches. That is why **Design2GarmentCode, ChatGarment, AIpparel, and NGL-Prompter** matter more for production than purely visual garment models. In contrast, **SewFormer, Panelformer, and NeuralTailor** are foundational but output lower-level edges, transforms, and stitches, which makes them better research baselines than final production systems. ¹

For a **local, backend-only Python/GPU stack**, the highest-confidence engineering recommendation is:

Use GarmentCode or a GarmentCode-like canonical pattern representation as the system of record, then build a local multimodal parser around it. Today, the best production-oriented references are **ChatGarment** for real-image robustness and valid JSON outputs, **Design2GarmentCode** for parametric program synthesis and structural correctness, and **NGL-Prompter** as the best recent design pattern for structured prompting, even though it is not yet production-ready. For fast generation or completion, **GarmentDiffusion** is the best current speed baseline. ²

The main research gap is not geometry alone. It is **manufacturing metadata**. Across the reviewed papers, representations and metrics focus on **panel geometry, 3D placement, stitch connectivity, simulation success, and Chamfer/F-score**, but they do **not** standardize **grainline, seam allowance, closure construction, ease policy, or notches** as first-class outputs or benchmarks. A production system therefore needs a **deterministic post-processor and validator** for those items. ³

Research landscape

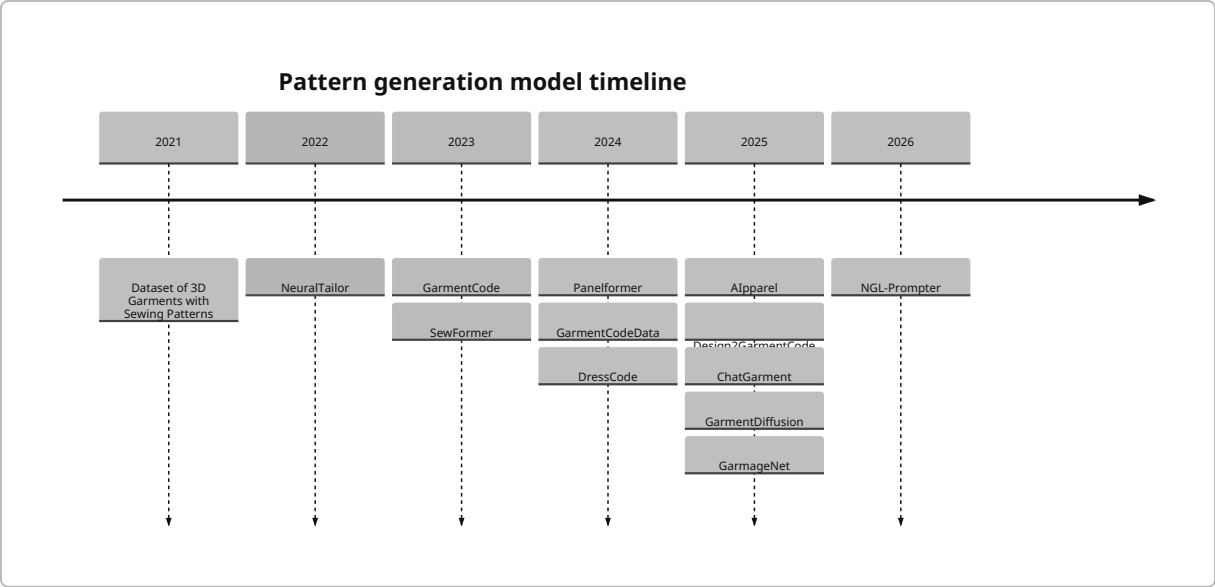
The modern line of work starts with public sewing-pattern data. The field moved from the earlier **Dataset of 3D Garments with Sewing Patterns** of roughly 20k design samples, to **NeuralTailor** for point-cloud-to-pattern reconstruction, then to **SewFactory** and **SewFormer** for single-image reconstruction at scale, and then to multimodal and language-centric systems such as **DressCode, Design2GarmentCode, ChatGarment, AIpparel, GarmentDiffusion, GarmageNet**, and **NGL-Prompter**. The most important enabling assets are now **GarmentCode, GarmentCodeData**, and multimodal annotations built on top of them. ⁴

Two representation families dominate:

Edge and stitch predictors. SewFormer, Panelformer, and NeuralTailor model garments as collections of panels made of Bezier or edge vectors, together with panel transforms and stitch relations. This is close to fabrication, but it is fragile because the model must get topology, panel closure, transforms, and seam pairing right in one shot. 5

Language or program mediated systems. Design2GarmentCode, ChatGarment, AIpparel, and NGL-Prompter shift the problem into a more semantic intermediate space: GarmentCode, JSON configuration, tokenized sewing patterns, or Natural Garment Language. This tends to improve validity and editability because the last stage is deterministic compilation rather than free-form geometry decoding. 6

That shift matters for engineering. If Maya must become a **backend pattern compiler and validator**, not just a renderer, then the second family aligns far better with your goal.



The most active researchers and clusters visible in the reviewed sources include **Maria Korosteleva and Olga Sorkine-Hornung** around GarmentCode and GarmentCodeData, the **Style3D and Zhejiang University** line around **Huamin Wang, Xiaogang Jin, Yong-Lu Li, Feng Zhou, Ruiyang Liu, and collaborators**, the **SewFormer** line around **Lijuan Liu, Xiangyu Xu, Zhijie Lin, Jiabing Liang, and Shuicheng Yan**, and the **ChatGarment / NGL** line around **Siyuan Bian, Yao Feng, Omid Taheri, and Michael Black**. Notable projects attached to those clusters are **GarmentCode, GarmentCodeData, Design2GarmentCode, ChatGarment, AIpparel, GarmageNet**, and **NGL-Prompter**. 7

Model assessment

Core comparison table

Model	Main task	Inputs	Outputs	Code and license	Key dependencies	Published hardware speed
Design2GarmentCode	Multimodal design to parametric pattern program	Text, image, sketch, mixed prompts	GarmentCode program, pattern JSON, optional 3D simulation	Official repo, MIT ⁸	Python 3.9.19, Torch 2.4.0, CUDA 12.1, Qwen2-VL-2B projector, optional NvidiaWarp GarmentCode ⁸	Training code-generated expert on a 5 GTX 4090 official inference latency published
ChatGarment	Estimation, generation, editing via VLM	Image, text, editing instruction	Garment JSON configuration, then GarmentCodeRC sewing patterns	Official repo, Apache-2.0 ¹⁰	torch 2.1.2, transformers 4.37.2, accelerate, bitsandbytes, gradio, deepspeed optional, flash-attn, GarmentCodeRC ¹¹	Trained on H100 reconstructions on A100 80GB 12.1s 3.5s GarmentCode + 33.9s stitch
NGL-Prompter	Training-free single-image pattern estimation	Image, optionally text	NGL attributes parsed deterministically into GarmentCode	Paper only in reviewed sources; code and data to be released for research use ¹³	Large VLMs such as GPT-5.0 and Qwen2.5-72B in the paper ¹⁴	No official throughput published accuracy comes from high VLM ¹⁵

Model	Main task	Inputs	Outputs	Code and license	Key dependencies	Publication hardware speed
AIpparel	Multimodal foundation model for sewing-pattern generation and editing	Text, image, sewing pattern editing	Generated sewing patterns that can be directly simulated	Official code repo, MIT ; weights and dataset on Hugging Face also MIT ¹⁶	Python 3.10, Torch 2.3.1, CUDA 12.1, transformers 4.31.0, svgpathtools ¹⁷	No official inference benchmarks; review source
GarmentDiffusion	Multimodal generative sewing-pattern model	Text, sketch image, incomplete pattern	Vectorized 3D sewing patterns from edge tokens	Official repo found; license not surfaced in fetched metadata ¹⁹	torch 2.6.0, cu118, diffusers 0.27, transformers 4.50.3, chamferdist, nvdiffrast, shapely ²⁰	Within second on single GPU; 100x faster than Sewing per pattern projected ²¹
SewFormer	Single-image reconstruction	Single RGB image	Panels, Bezier edges, 3D transforms, stitches	Official repo; license not clearly surfaced in fetched metadata ²²	Conda env, checkpoint, SewFactory dataset, simulation path through Maya + mayapy ²³	Trained epoch A100 ; no official inference timing
Panelformer	2D garment image to sewing pattern	Rendered front-view garment image	Panel edge vectors, curvatures, quaternion rotation, translation, stitches	Official repo, MIT ²⁵	PyTorch implementation; built on NeuralTailor and AnchorUDF preprocessing ²⁶	Single 3090 2 input 512x512 official inference timing

Model	Main task	Inputs	Outputs	Code and license	Key dependencies	Published hardware speed
NeuralTailor	3D point cloud to pattern	Garment point cloud	Sewing-pattern structure and stitches	Official repo; installation docs provided, license not clearly surfaced in fetched metadata ²⁸	Python 3.9, PyTorch, PyG, libigl, custom Garment-Pattern-Generator package ²⁹	Full training report to 4 d dependencies hardware official inference timing
GarmageNet	Multimodal pattern design plus generic 3D garment modeling	Text, sketch, point cloud, unconditional	Garmage panels plus stitching via GarmageJigsaw and 3D initialization	Official repo, CC BY-NC-ND 4.0 ³¹	Ubuntu 22.04, CUDA 11.8, Python 3.10, PyTorch 2.6, diffusers, chamferdist, nvdiffrast ³²	Training example exposed GPU friendly large settings official inference benchmark source review

What each model is actually good at

Design2GarmentCode is the cleanest reference for **production-style structured generation**. It translates multimodal concepts into **GarmentCode-compliant programs**, uses validation loops to keep answers in valid parameter space, and reports strong generation quality with **100% SSR for text-guided generation** and **94% SSR for image-guided generation**, well ahead of DressCode and SewFormer in their comparison. Its main weakness is deployment friction: the released implementation defaults to an **OpenAI-compatible MMUA API** and uses a local projector plus Qwen2-VL path, so a strictly offline deployment means replacing the external agent with a self-hosted VLM endpoint. It is also limited by **one-to-one stitching relationships** and the underlying design space of GarmentCode. ³⁴

ChatGarment is the best current reference if the priority is **real-image robustness** rather than synthetic benchmark neatness. It fine-tunes **LLaVA-1.5 7B**, outputs a **JSON sewing-pattern configuration**, reports **zero failure rate** on Dress4D and CloSe in its main comparisons, and explicitly supports image reconstruction, text generation, and text editing. The downside is operational cost: the authors' speed breakdown shows the main bottleneck is not the VLM itself but the **Warp-based sewing pattern stitching stage**, which dominates total latency. That makes ChatGarment strong for backend batch jobs, but heavy for interactive UI unless you redesign the stitching stage. ³⁵

NGL-Prompter is the most important new research idea in this space. It shows that **structured prompting plus deterministic parsing** can outperform fine-tuned garment VLM systems on **Dress4D, CloSe, and ASOS_5K**, including **multi-layer outfits**, without task-specific training. That is a big deal. But it is not yet a safe production choice for your stack because the reviewed sources show **research-use release pending**, and the paper evaluates with very large models like **Qwen2.5-72B** and **GPT-5.0**, which are costly or awkward to self-host in a small local deployment. Treat it as a design template, not a drop-in backend. ³⁶

AIpparel is promising because it sits between ChatGarment and Design2GarmentCode. It is a multimodal foundation model trained on **over 120,000 unique garments**, with code and weights released under **MIT**, and it is built explicitly for sewing-pattern prediction and editing. In a local deployment, AIpparel is attractive because it avoids the heavier multi-stage stitching bottleneck documented by ChatGarment and is directly aligned with GarmentCodeData. What is still missing in the reviewed sources is a detailed real-image deployment benchmark like ChatGarment or NGL-Prompter provide. ³⁷

GarmentDiffusion is the fastest modern generative option. It supports **text, image sketch, and incomplete-pattern completion**, uses compact edge-token representations, and reports **sub-second generation on one A10 GPU** plus a large speedup over SewingGPT. It also scales to **GarmentCodeData**, which matters for complex garments. The caution is that its output remains an **edge-token pattern representation**, not a semantic garment DSL, and the paper itself notes that current annotations still lack enough edge and panel connectivity information to fully eliminate simulation failures. That makes it excellent as a generator or completer, but weaker as the canonical source of truth than GarmentCode-based methods. ³⁸

SewFormer is still the best fully open **image-to-panel baseline**. It introduced **SewFactory**, a large image-pattern dataset, and reports strong quantitative results with **95.7% stitch F1** on SewFactory. But later work explicitly highlights its failure modes: **self-intersecting panels**, wrong cross-garment stitches, and weaker alignment to prompts than newer language-mediated systems. In practice, use SewFormer as a teacher, baseline, or auxiliary reconstructor, not as your primary production compiler. ³⁹

Panelformer is useful if you want a lighter, MIT-licensed panel/stitch predictor from **2D garment images**, but it is trained and evaluated on **rendered front views** from the Korosteleva dataset rather than unconstrained clothed-human photos. It predicts panel outlines, rotations, translations, and stitch probabilities, and it runs on a **single RTX 3090 24GB**, which is attractive. Still, it is not the strongest choice for your intended backend because your problem is not just panel reconstruction. It is full semantic garment compilation for sewable output. ⁴⁰

NeuralTailor matters because it established much of the modern evaluation and representation language for pattern reconstruction, and the repo is well documented for local experimentation. But it is fundamentally a **3D point-cloud-to-pattern** system, not an image-to-pattern system. If your frontend already has 3D garment scans or a trusted geometry reconstruction stage, then NeuralTailor is still relevant. Otherwise, it is one layer too low for your target product. ⁴¹

Decision matrix

Model scoring for your use case

Scores are relative and tuned to your stated constraints: **local server-side deployment, metric sewing patterns, backend integration, and sewability over visual novelty.**

Model	Accuracy	Sewability	Parametric output	Ease of integration	Code quality	License fit	Recommended use
Design2GarmentCode	4	5	5	4	4	5	Best architectural reference for production
ChatGarment	5	4	5	3	4	4	Best current local-image backbone
NGL-Prompter	5	4	5	1	1	1	Research only for now
AIpparel	4	4	4	4	4	5	Strong local candidate
GarmentDiffusion	4	3	3	4	4	2	Fast generator and completer
SewFormer	3	2	2	3	3	2	Benchmark baseline only
Panelformer	3	2	2	3	3	5	Secondary baseline
NeuralTailor	3	3	2	2	3	2	Only if 3D input exists
GarmageNet	4	4	4	3	4	1	Research only, noncommercial

These scores follow directly from the reviewed output representations, benchmarks, and repo status. GarmentCode and JSON-based systems rate higher on **sewability** because they compile from structured design spaces, while edge-token or panel-only systems rate lower because they leave more room for topological failure, invalid seams, or fabrication metadata gaps. ⁴²

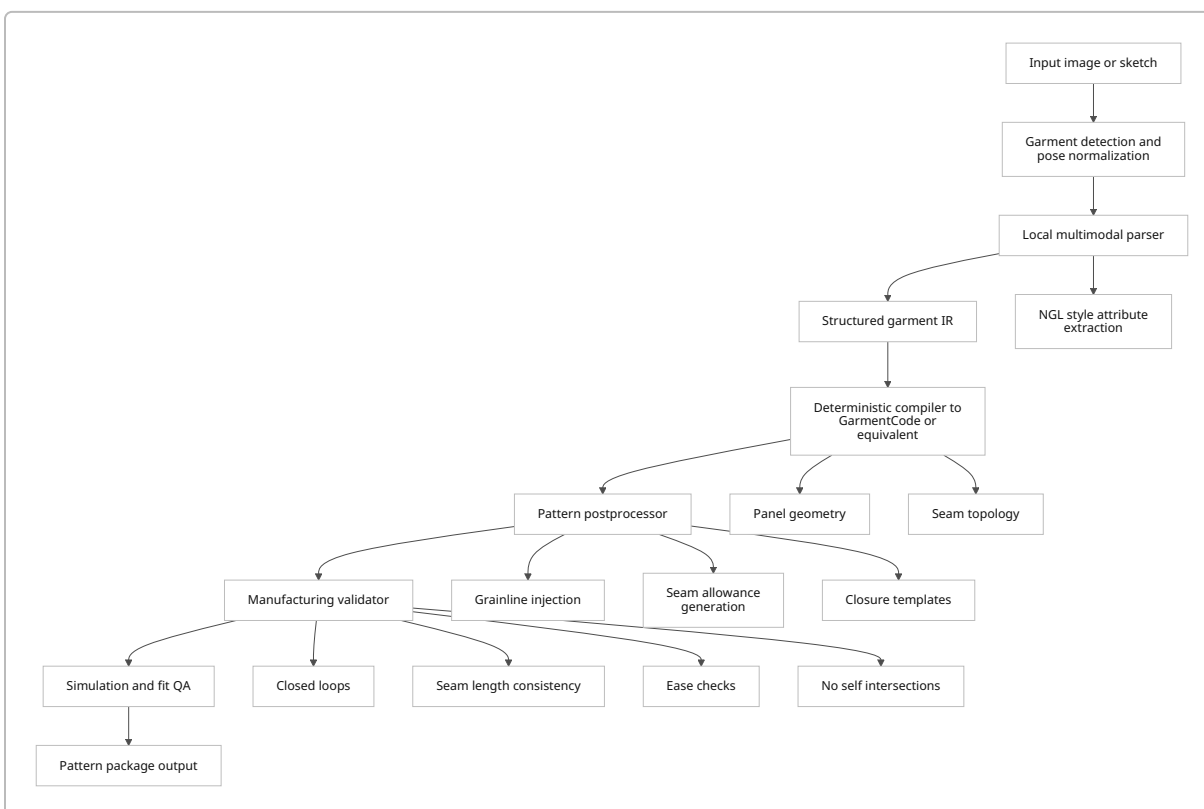
Straight recommendation

If you need one answer:

Build around **GarmentCode** as the canonical representation. Use a **ChatGarment** or **Aipparel** style local parser now, borrow **NGL-Prompter**'s structured prompting ideas, and keep **GarmentDiffusion** only as a speed-oriented auxiliary module for completion or variation generation. Do not make **SewFormer** or **Panelformer** your production core. ⁴³

Recommended backend architecture

The production architecture should separate **semantic understanding**, **pattern compilation**, and **manufacturing validation**. That is the only way to satisfy sewability and local deployment at the same time.



Recommended stack

A practical stack for your private backend is:

Canonical pattern engine. Use **GarmentCode** and **GarmentCodeData** as the canonical representation and training substrate. That gives you parametric structure, metric precision, simulation hooks, and a large public dataset of made-to-measure garments. ⁴⁴

Local parser. Start with **ChatGarment** or **Aipparel** style local multimodal parsing. If you can host 7B to 13B comfortably, that is realistic. If you can host 72B across larger local infrastructure, borrow **NGL-Prompter** prompting strategies, but do not wait for that to ship your backend. ⁴⁵

Compiler and refiner. Use a deterministic compiler that converts structured JSON or NGL-style attributes into GarmentCode parameters, then run a local numeric refiner for body measurements, symmetry, and seam-length balancing. This is the architectural lesson from Design2GarmentCode and NGL-Prompter. ⁴⁶

Fast generative helper. Add **GarmentDiffusion** only if you need one of three things: fast sketch-conditioned generation, pattern completion from partial patterns, or diversity sampling for design exploration. ²¹

Estimated implementation roadmap

Phase	Deliverable	Required infra	Estimated dev time	Main risks
Canonical pattern layer	GarmentCode-based internal schema, import/export, simulation hooks	Python, FastAPI, local GPU box, GarmentCode stack	2 to 3 weeks	Schema drift between model outputs and compiler
Local parser MVP	Image to structured JSON using ChatGarment or AIpparel-style model	1× A100 or 1× RTX 4090 for inference, model serving	3 to 5 weeks	Latency, model hallucination on rare fashion details
Deterministic compiler	JSON to panels, stitches, body sizing, draft rules	Python geometry library, pattern compiler tests	2 to 4 weeks	Invalid topology on edge cases
Manufacturing postprocessor	Grainline, seam allowance, notches, closure templates	Rule engine, pattern library	2 to 3 weeks	Research models do not supervise these attributes
Validation harness	Seam checks, loop closure, ease checks, simulation SSR	Warp or C-IPC or GarmentCode simulator	2 to 4 weeks	Simulation bottleneck and false negatives
Quality loop	Human review tooling and regression suite	Internal HTML dashboard, storage, render farm optional	2 weeks	Hard to label “good enough” without domain expert rubrics

Throughput and cost view

Published speeds are sparse, but the direction is clear. **ChatGarment** is accurate on real data but relatively slow because stitching dominates runtime, while **GarmentDiffusion** is dramatically faster and **Design2GarmentCode** shifts cost into semantic reasoning rather than long autoregressive edge generation. ⁴⁷

My engineering estimate for local deployment is:

Model	Practical local floor	Estimate for throughput
Design2GarmentCode	24GB class GPU plus self-hosted OpenAI-compatible VLM endpoint	Roughly 5 to 20s per sample if fully local, depending on local VLM and simulation
ChatGarment	40GB to 80GB class GPU preferred	Roughly 50s per sample on A100 according to the paper; the stitching stage is the bottleneck
AIpparel	24GB to 48GB class GPU	Likely single-digit to low tens of seconds per sample, but not formally reported
NGL-Prompter with Qwen-72B	Multi-GPU or very high-memory host	Likely too heavy for a single 24GB box ; better as research infrastructure
GarmentDiffusion	24GB class GPU easily sufficient	Sub-second to ~1s per sample for pure generation, excluding external simulation
SewFormer and Panelformer	24GB class GPU	Usually sub-second to a few seconds for forward inference, excluding simulation

Those are engineering estimates where official numbers are absent. The published numbers are the ones to trust first.

Validation and minimal reproducible test plan

What to validate that papers usually do not

A production pattern engine should add five validations that current academic benchmarks do not standardize:

Seam topology. Every stitch pair must be bijective, cross-panel, and length-compatible after ease policy. Reject orphan seams, self-seams, and seam chains with inconsistent orientation.

Closed-loop geometry. Every panel must be a valid closed polygon or spline loop with no self-intersections and consistent winding.

Grainline and seam allowance. These should be added deterministically after geometry generation. In the reviewed research, they are not first-class evaluation targets, so you should not expect the models to learn them reliably. ⁴⁸

Ease. Measure garment circumference and lengths against the target body block and category policy. A skirt, fitted bodice, and hoodie should not share the same tolerance logic.

Closures. Zippers, plackets, buttons, elastic waists, and invisible closure assumptions need explicit templates. Current benchmarks mostly score geometry and reconstruction quality, not closure correctness.

49

Minimal reproducible test plan

Start small and brutal:

Test slice	Count	Goal
Simple tops	15	Sleeve, neckline, fit, front-back consistency
Pants	10	Rise, inseam, waistband, leg symmetry
Skirts	10	Waist ease, hem style, panel closure
Dresses	10	Multi-panel topology, bodice-skirt coherence
Outerwear	5	Collar, open front, closure support

For each sample, record:

Check	Pass target
Panel loop validity	100%
No self-intersections	100%
Seam bijection validity	100%
Seam-length mismatch before allowance	median under 1% or under 3 mm
Simulation success rate	over 95%
Manual style agreement	over 4 out of 5
Grainline present	100%
Seam allowance present	100%
Closure metadata present when needed	100%

Recommended experiments

The highest-value experiments, in order, are:

Compiler-first ablation. Feed the same image set through ChatGarment, AIpparel-style parsing, and a hand-built NGL prompt pipeline, but compile all three into the same internal pattern schema. This isolates the parser from the compiler.

Topological stress set. Create a small internal benchmark of garments with hard seam graphs: asymmetric skirts, layered dresses, jackets with open fronts, waists with gathers, and garments with occluded sleeves. Later papers repeatedly show failures on intricate details, unseen relations, or multi-layer setups. ⁵⁰

Measurement fidelity test. On a subset with known target body measurements, compare final waist, chest, hip, inseam, and garment length against intended dimensions after postprocessing.

Closure test. Add a deterministic closure generator and compare before and after simulation validity.

The minimal success condition for moving past prototype stage is simple: **no invalid seams, no self-intersecting panels, and no simulation failures on the simple category slice.**

Reading list and ecosystem

Prioritized reading list

GarmentCode: Programming Parametric Sewing Patterns. The most important paper if you care about production. It gives you the actual parametric language that later systems target or implicitly emulate. ⁵¹

GarmentCodeData. The most important dataset paper for modern pattern generation because it provides scale, body variation, and a public generation pipeline. ⁵²

NeuralTailor. Foundational for structure recovery and still useful for understanding the classical metric stack: Panel L2, transforms, stitch quality, unseen topology evaluation. ⁵³

SewFormer. The key image-to-pattern baseline. Read it to understand what edge-level reconstruction does well and where it breaks. ⁵⁴

Design2GarmentCode. Best paper to understand why **program synthesis** is a better production abstraction than free-form edge generation. ⁵⁵

ChatGarment. Best paper to understand multimodal local VLM parsing, image robustness, and real latency bottlenecks. ⁵⁶

NGL-Prompter. Best current research direction for prompt-engineered structured parsing with deterministic compilation. ⁵⁷

GarmentDiffusion. Best fast generator reference if you need completion, diversity, or sketch-conditioned sampling. ²¹

AIpparel. Best current open multimodal foundation-model candidate on top of the GarmentCode ecosystem. ¹⁸

GarmageNet. Read this if you want the next generation beyond pure pattern generation, especially if 3D initialization and downstream simulation-ready assets matter. Be aware of its noncommercial license. ⁵⁸

Open questions and limitations

Some things are still unresolved in the public research:

The license status for some repos, especially **SewFormer**, **NeuralTailor**, and **GarmentDiffusion**, was not clearly surfaced in the fetched metadata, so treat them as requiring manual legal review before product use. ⁵⁹

Several strong systems still depend on external or very large models. **Design2GarmentCode** defaults to an API-based MMUA path, and **NGL-Prompter** is evaluated with GPT-5.0 and Qwen2.5-72B. That is workable for research, but not ideal for a tight local-only product unless you invest in model serving infrastructure. ⁶⁰

The field still lacks first-class evaluation for **grainline, seam allowance, notches, and closure construction**. In other words, academic SOTA is not yet manufacturing-complete. Your validator layer is not optional.

Bottom line: for Maya's backend vision, do **not** build around raw image-to-panel predictors alone. Build a **GarmentCode-centered compiler stack**, use **ChatGarment** or **AIpparel** as the local parser, adopt **NGL-style structured prompting**, add a **deterministic manufacturing postprocessor**, and use **GarmentDiffusion** only where speed or completion matters. That is the shortest path to a private system that outputs patterns you can actually trust.

¹ ⁵⁵ Turning Design Concepts to Tangible Garments Through ...

https://arxiv.org/html/2412.08603v3?utm_source=chatgpt.com

² ¹² ³⁵ ⁴⁵ ⁴⁷ ⁴⁹ <https://arxiv.org/html/2412.17811v3>

<https://arxiv.org/html/2412.17811v3>

³ ²⁷ ⁴⁸ <https://openaccess.thecvf.com/content/WACV2024/papers/>

Chen_Panelformer_Sewing_Pattern_Reconstruction_From_2D_Garment_Images_WACV_2024_paper.pdf

<https://openaccess.thecvf.com/content/WACV2024/papers/>

Chen_Panelformer_Sewing_Pattern_Reconstruction_From_2D_Garment_Images_WACV_2024_paper.pdf

⁴ <https://github.com/maria-korosteleva/Garment-Pattern-Generator>

<https://github.com/maria-korosteleva/Garment-Pattern-Generator>

⁵ ²⁴ ³⁹ ⁵⁴ <https://ar5iv.labs.arxiv.org/html/2311.04218>

<https://ar5iv.labs.arxiv.org/html/2311.04218>

⁶ ⁸ ³⁴ ⁴² ⁴⁶ ⁶⁰ GitHub - Style3D/design2garmentcode-impl: Official implementation for Design2GarmentCode: Turning Design Concepts to Tangible Garments Through Program Synthesis (CVPR 2025) · GitHub

<https://github.com/Style3D/design2garmentcode-impl>

⁷ ⁵² <https://arxiv.org/abs/2405.17609>

<https://arxiv.org/abs/2405.17609>

⁹ ⁵⁰ Design2GarmentCode: Turning Design Concepts to Tangible Garments Through Program Synthesis

<https://arxiv.org/html/2412.08603v3>

10 GitHub - biansy000/ChatGarment: [CVPR'25] ChatGarment: Garment Estimation, Generation and Editing via Large Language Models · GitHub

<https://github.com/biansy000/ChatGarment>

11 <https://github.com/biansy000/ChatGarment/blob/main/pyproject.toml>

<https://github.com/biansy000/ChatGarment/blob/main/pyproject.toml>

13 36 57 <https://arxiv.org/abs/2602.20700>

<https://arxiv.org/abs/2602.20700>

14 15 <https://arxiv.org/html/2602.20700v1>

<https://arxiv.org/html/2602.20700v1>

16 17 <https://github.com/Sayvai-io/Aipparel>

<https://github.com/Sayvai-io/Aipparel>

18 37 <https://arxiv.org/abs/2412.03937>

<https://arxiv.org/abs/2412.03937>

19 <https://github.com/Shenfu-Research/Garment-Diffusion>

<https://github.com/Shenfu-Research/Garment-Diffusion>

20 <https://github.com/Style3D/garmagenet-impl/blob/main/requirements.txt>

<https://github.com/Style3D/garmagenet-impl/blob/main/requirements.txt>

21 38 <https://arxiv.org/html/2504.21476v1>

<https://arxiv.org/html/2504.21476v1>

22 23 59 <https://github.com/sail-sg/sewformer>

<https://github.com/sail-sg/sewformer>

25 26 40 <https://github.com/ericsujw/Panelformer>

<https://github.com/ericsujw/Panelformer>

28 41 53 <https://arxiv.org/abs/2201.13063>

<https://arxiv.org/abs/2201.13063>

29 <https://github.com/maria-korosteleva/Garment-Pattern-Estimation/blob/master/docs/Installation.md>

<https://github.com/maria-korosteleva/Garment-Pattern-Estimation/blob/master/docs/Installation.md>

30 <https://github.com/maria-korosteleva/Garment-Pattern-Estimation/blob/master/docs/Running.md>

<https://github.com/maria-korosteleva/Garment-Pattern-Estimation/blob/master/docs/Running.md>

31 32 33 <https://github.com/Style3D/garmagenet-impl>

<https://github.com/Style3D/garmagenet-impl>

43 51 GarmentCode: Programming Parametric Sewing Patterns

https://dl.acm.org/doi/10.1145/3618351?utm_source=chatgpt.com

44 GarmentCode: Programming Parametric Sewing Patterns

https://github.com/maria-korosteleva/GarmentCode?utm_source=chatgpt.com

56 <https://arxiv.org/abs/2412.17811>

<https://arxiv.org/abs/2412.17811>

58 <https://arxiv.org/html/2504.01483v3>

<https://arxiv.org/html/2504.01483v3>